

MACHINE LEARNING ENGINEERING

FASE 5 | AULA 07 -

MONITORAMENTO E MANUTENÇÃO DE APIS

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	5
O QUE VOCÊ VIU NESTA AULA?	36
REFERÊNCIAS.....	37

EMSE

O QUE VEM POR AÍ?

Nesta aula, vamos dar o próximo passo natural depois de colocar uma API de ML em produção: garantir que ela continue saudável ao longo do tempo. Saímos da fase “funciona” e entramos na fase “funciona sempre, com qualidade”. Você vai ver como usar logs estruturados, métricas de desempenho e ferramentas como Prometheus e Grafana para enxergar o que está acontecendo dentro da sua API em tempo real, em vez de “torcer para dar certo”.

Também vamos discutir alertas inteligentes, versionamento e atualização de modelos, além de rotinas de backup e manutenção periódica. A ideia é fechar o ciclo de MLOps: não basta treinar e expor um modelo, é preciso monitorar, medir, versionar e corrigir continuamente. Ao final da aula, você terá uma visão prática de como operar sua API de inferência como um serviço profissional, com observabilidade, planos de resposta e evolução controlada em produção.

HANDS ON

Prepare-se para mais uma aula hands on, onde vamos trazer ferramentas de métricas e desempenho para nossa API!

EXEMPLO

SAIBA MAIS

Monitoramento e Manutenção de APIs de ML em Produção

A nossa API está no ar, e agora? Manter um modelo de Machine Learning em produção requer observabilidade contínua, ou seja, monitoramento ativo e manutenção para garantir desempenho e confiabilidade. As grandes empresas que vimos nos cases como Amazon e Google enfatizam que o monitoramento é tão crucial quanto segurança para o sucesso de sistemas em produção.

Nesta aula, vamos consolidar práticas de como monitorar e manter APIs de ML em produção de forma eficaz.

Logs Estruturados

Os logs são o histórico de tudo que acontece na nossa API e são essenciais para auditoria, depuração de bugs e até detecção automatizada de anomalias. No entanto, logs soltos em textos simples no terminal dificultam nossas análises. A recomendação é usar logs estruturados tipicamente em JSON com campos padronizados.

Quando adotamos esse esquema consistente do JSON, facilitamos o parsing e a correlação de eventos, além de reduzir ruído e melhorar análises automatizadas.

Com esse log estruturado, podemos filtrar rapidamente por campos como ID de usuário, endpoint ou código de status, algo que seria impraticável em logs de texto puro como vimos antes. E claro, como temos essa formatação consistente, conseguimos usar ferramentas de segurança e auditoria para nos apoiar no monitoramento da nossa API.

Boas Práticas de Logging e Centralização

Para um monitoramento eficaz de APIs, a adoção de boas práticas de logging é fundamental. Os logs devem ser enriquecidos com metadados relevantes, incluindo:

- **Informações Essenciais:** timestamp (data/hora), level (nível de severidade como INFO, WARN, ERROR), request_id (identificador único da requisição), client_ip e informações de usuário/autenticação.
- **Detalhes do Acesso:** o endpoint acessado e uma mensagem de contexto clara.

É crucial evitar o registro de dados pessoais sensíveis (PII), em conformidade com regulamentações como LGPD e GDPR.

A centralização dos logs em um repositório unificado é essencial para facilitar buscas e auditorias em todos os serviços. Plataformas de logging gerenciadas (como AWS CloudWatch Logs, Loggly, Papertrail) ou stacks open-source (ELK/ElasticSearch) são ideais para receber e indexar logs.

Ao enviar logs no formato JSON (logs estruturados), esses sistemas podem automaticamente parsear os campos, permitindo consultas estruturadas e análises quase em tempo real.

Exemplo de Log Estruturado (JSON):

```
{  
  "timestamp": "2025-12-04T17:40:23Z",  
  "level": "INFO",  
  "service": "modelo-fraude-API",  
  "endpoint": "/api/v1/predict",  
  "client_ip": "203.0.113.42",  
  "request_id": "58d9c96e-ae9f-43db-a353-c48e7a70bfa8",  
  "message": "Predição realizada com sucesso",  
  "latency_ms": 120  
}
```

Métricas da API

Além do registro de eventos (logs), o monitoramento da saúde e performance de uma API exige o acompanhamento de métricas quantitativas fundamentais.

MÉTRICA	DESCRIÇÃO E IMPORTÂNCIA
Latência de Resposta	O tempo necessário para processar as requisições. É vital monitorar a latência média e, crucialmente, percentis altos como o p95 (95º percentil). O p95 reflete a experiência dos piores 5% dos usuários, sendo mais eficaz que a média para identificar outliers de performance. A meta é definir e manter SLOs (Objetivos de Nível de Serviço) de latência, garantindo que o p95, por exemplo, permaneça abaixo de um limite aceitável.
Throughput (Vazão)	O volume de tráfego que a API consegue lidar, geralmente medido em Requisições por Segundo (RPS) ou por Minuto (RPM). Indica a capacidade e a carga do sistema. O monitoramento de throughput ajuda a: detectar gargalos e padrões de uso; sinalizar instabilidades (declínio súbito) ou picos (ataque ou sucesso); informar o contexto de outras métricas (um aumento de latência pode ser esperado em caso de saturação (throughput máximo)); acionar alertas e autoescalonamento ao ultrapassar limites.
Taxa de Erros (5xx)	A porcentagem de requisições que resultam em códigos de erro de servidor (HTTP 5xx). É o indicador mais direto de falha interna. Idealmente deve ser 0% em produção. É importante focar nos erros 5xx (falhas do servidor) e não nos 4xx (erros do cliente). Thresholds para alertas devem ser rigorosos (ex.: > 1% pode ser crítico). Seu acompanhamento revela regressões introduzidas por deployments ou instabilidades intermitentes.
(Opcional) Acurácia do Modelo	Aplicável a APIs de Machine Learning. Se houver um mecanismo de feedback ou comparação com o ground truth, é possível medir o desempenho do modelo em produção (ex.: taxa de acerto, relevância da recomendação). Uma queda na acurácia indica drift de conceito (o modelo perde validade com novos dados), sinalizando a necessidade de re-treinamento e atualização.

Tabela 1 - Monitoramento de Métricas Essenciais para APIs
Fonte: Elaborado pelo autor (2026)

Outras métricas podem incluir uso de recursos (CPU, memória), número de usuários ativos ou fila de requisições. No entanto, latência, throughput e taxa de erros são a base para a confiabilidade e desempenho de qualquer API.

A Importância do Conjunto de Métricas

Essas métricas combinadas oferecem visibilidade total: latência e throughput refletem experiência e capacidade, enquanto erros 5xx refletem confiabilidade. Por exemplo, a elevação simultânea do p95 de latência e da taxa de erro sinaliza degradação ou um bug crítico sob carga, permitindo uma investigação proativa.

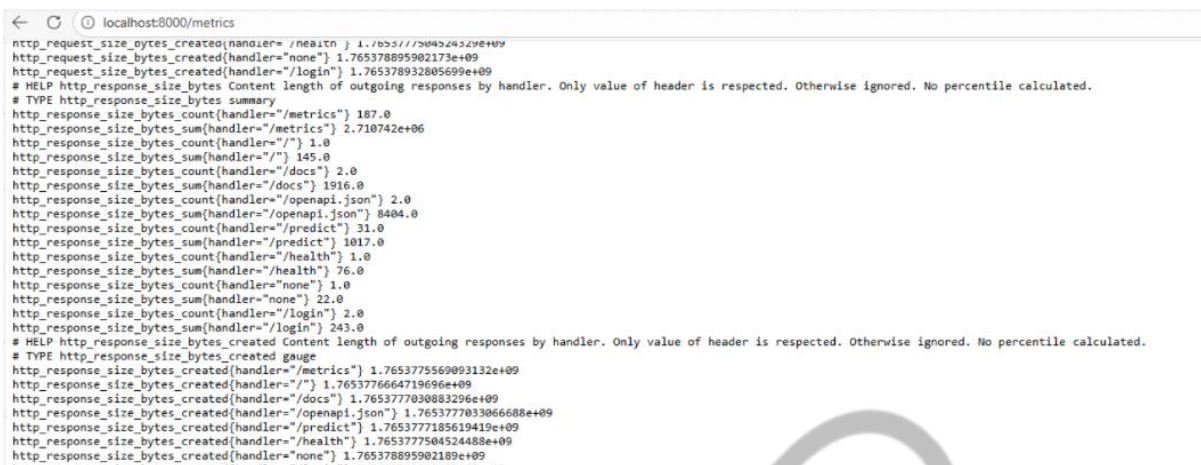
É essencial estabelecer baseline (valores normais típicos) e thresholds (limites de alerta) para cada métrica. Se o p95 de latência normal for 300ms e saltar para 800ms, ou se o throughput usual de 50 RPS pular para 200 RPS, um alerta deve ser acionado para investigação.

A coleta dessas métricas geralmente envolve a instrumentação do código da API com contadores e temporizadores ou o uso de middlewares que registram metadados de cada requisição (endpoint, duração, código de resposta). Ferramentas especializadas, como o Prometheus, facilitam esse processo.

O monitoramento eficaz de métricas em tempo real é crucial e, para isso, ferramentas especializadas são frequentemente adotadas. O Prometheus, um sistema open-source de monitoramento inicialmente desenvolvido pela SoundCloud, destaca-se por sua capacidade de coletar e armazenar métricas numéricas de sistemas e aplicações em um banco de dados de séries temporais (time series). Ele permite consultas e cálculos complexos sobre esses dados.

Em complemento, o Grafana atua como uma plataforma aberta de visualização. Ele se conecta ao Prometheus para criar dashboards interativos e painéis visuais. Em essência, o Prometheus instrumenta a API (coleta e guarda as métricas), enquanto o Grafana exibe visualmente essas informações em gráficos e painéis.

A integração do Prometheus com APIs, como as desenvolvidas em FastAPI (Python), é direta. O Prometheus opera em um modelo pulling (puxando): a própria API expõe um endpoint (geralmente /metrics) que retorna as métricas atuais em formato de texto (formato Prometheus).



```

localhost:8000/metrics
http_request_size_bytes_created(handler="/health") 1.7653775569093132e+09
http_request_size_bytes_created(handler="/metrics") 1.7653775569093132e+09
http_request_size_bytes_created(handler="/login") 1.7653775569093132e+09
# HELP http_response_size_bytes Content length of outgoing responses by handler. Only value of header is respected. Otherwise ignored. No percentile calculated.
# TYPE http_response_size_bytes summary
http_response_size_bytes_count(handler="/metrics") 187.0
http_response_size_bytes_sum(handler="/metrics") 2.710742e+06
http_response_size_bytes_count(handler="/") 1.0
http_response_size_bytes_sum(handler="/") 145.0
http_response_size_bytes_count(handler="/docs") 2.0
http_response_size_bytes_sum(handler="/docs") 1916.0
http_response_size_bytes_count(handler="/openapi.json") 2.0
http_response_size_bytes_sum(handler="/openapi.json") 8404.0
http_response_size_bytes_count(handler="/predict") 31.0
http_response_size_bytes_sum(handler="/predict") 1017.0
http_response_size_bytes_count(handler="/health") 1.0
http_response_size_bytes_sum(handler="/health") 76.0
http_response_size_bytes_count(handler="/none") 1.0
http_response_size_bytes_sum(handler="/none") 22.0
http_response_size_bytes_count(handler="/login") 2.0
http_response_size_bytes_sum(handler="/login") 243.0
# HELP http_response_size_bytes_created Content length of outgoing responses by handler. Only value of header is respected. Otherwise ignored. No percentile calculated.
# TYPE http_response_size_bytes_created gauge
http_response_size_bytes_created(handler="/metrics") 1.7653775569093132e+09
http_response_size_bytes_created(handler="/") 1.7653775569093132e+09
http_response_size_bytes_created(handler="/docs") 1.7653775569093132e+09
http_response_size_bytes_created(handler="/openapi.json") 1.7653775569093132e+09
http_response_size_bytes_created(handler="/predict") 1.7653775569093132e+09
http_response_size_bytes_created(handler="/health") 1.7653775569093132e+09
http_response_size_bytes_created(handler="/none") 1.7653775569093132e+09

```

Figura 1 - /metrics
Fonte: Elaborado pelo autor (2026)

O servidor Prometheus, então, periodicamente (e.g., a cada 15 segundos) faz o scrape (coleta) desse endpoint e armazena os valores. Para isso, a biblioteca prometheus-client em Python facilita a criação de objetos para diferentes tipos de métricas (como Counter, Gauge, Histogram) e oferece um aplicativo ASGI para expor os resultados.

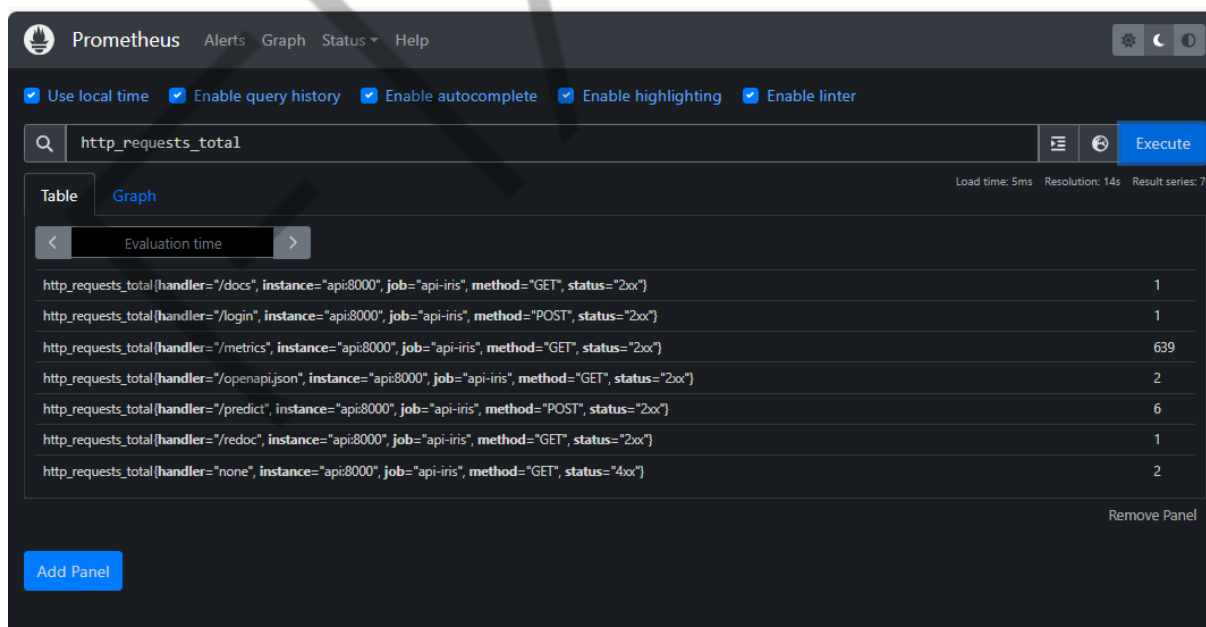


Figura 2 – Prometheus
Fonte: Elaborado pelo autor (2026)

A visualização no Grafana é a seguinte:

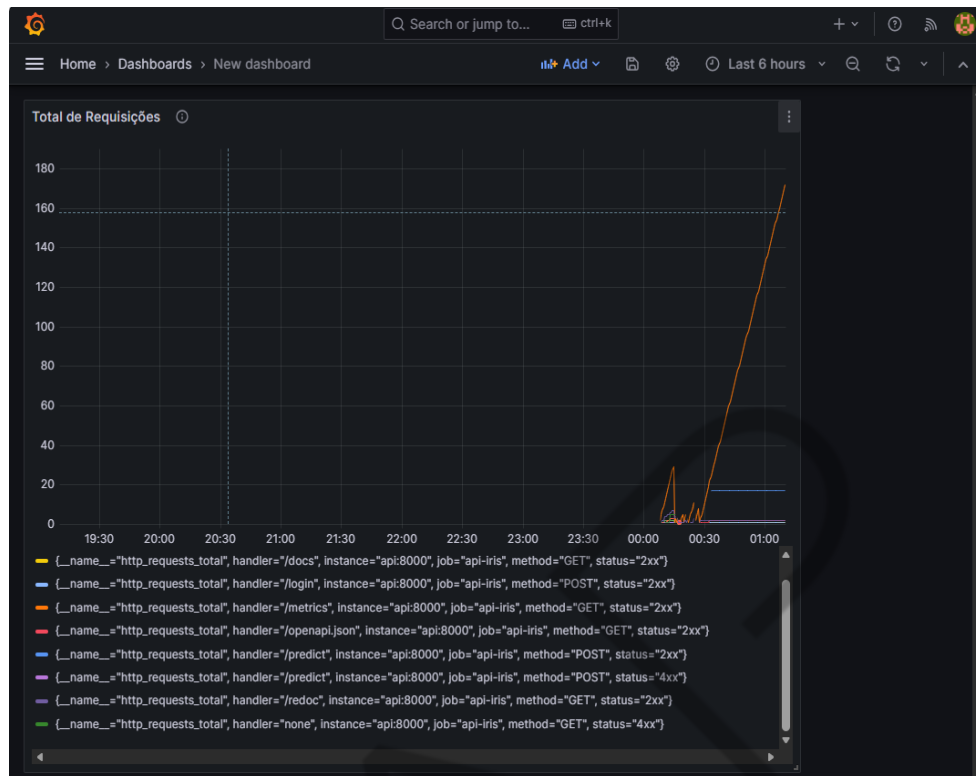


Figura 3 - Exemplo Grafana
Fonte: Elaborado pelo autor (2026)

Uma vez que o Prometheus está capturando as métricas, o Grafana é configurado para utilizá-lo como fonte de dados. No Grafana, é possível construir diversos tipos de visualizações, como:

- Gráficos de séries temporais para acompanhar a latência média ou p95 ao longo do tempo.
- Gráficos da taxa de erros 5xx por minuto.
- Gráficos do throughput (RPS - Requisições por Segundo) durante o dia.
- Painéis que exibem números atuais (gauges, counters), como o total de requisições desde o último deploy.

Hands On - Implementação de Monitoramento

Vamos criar um módulo de configuração de logs que formata tudo em JSON.

Primeiro, precisamos instalar a biblioteca: `install python-json-logger`

```
"""
```

```
Configuracao de Logs Estruturados (JSON)
```

```

Facilita analise em ferramentas como CloudWatch, Kibana,
Loggly
"""
import logging
import sys
from datetime import datetime
from pythonjsonlogger import jsonlogger

class CustomJsonFormatter(jsonlogger.JsonFormatter):
    """
    Formatter customizado que adiciona campos extras a cada
log.
    """
    def add_fields(self, log_record, record,
message_dict):
        super().add_fields(log_record, record,
message_dict)
        # Campos padrao em todo log
        log_record['timestamp'] =
datetime.utcnow().isoformat() + 'Z'
        log_record['level'] = record.levelname
        log_record['service'] = 'api-iris'
        log_record['logger'] = record.name
        # Garante que message existe
        if not log_record.get('message'):
            log_record['message'] = record.getMessage()

def setup_logging(level: str = "INFO") -> logging.Logger:
    """
    Configura e retorna um logger com formato JSON.

    Args:
        level: Nivel minimo de log (DEBUG, INFO, WARNING,
ERROR)

    Returns:
        Logger configurado
    """
    logger = logging.getLogger("api")
    logger.setLevel(getattr(logging, level.upper()))
    # Handler para stdout (container-friendly)
    # Containers capturam stdout/stderr automaticamente
    handler = logging.StreamHandler(sys.stdout)
    handler.setFormatter(CustomJsonFormatter(
        '%(timestamp)s %(level)s %(name)s %(message)s'
    ))
    # Remove handlers anteriores (evita duplicacao)
    logger.handlers = []
    logger.addHandler(handler)
    logger.propagate = False
    return logger
# Logger global para importar em outros modulos

```

```
logger = setup_logging()
```

Código-fonte 1 – Módulo de configuração de logs que formata tudo em JSON

Fonte: Elaborado pelo autor (2026)

Agora vamos criar um middleware que automaticamente loga todas as requisições:

```
"""
Middlewares para Logging e Metricas
Intercepta todas as requisicoes para logging automatico
"""
import time
import uuid
from fastapi import Request
from starlette.middleware.base import BaseHTTPMiddleware
from app.logging_config import logger

class LoggingMiddleware(BaseHTTPMiddleware):
    """
    Middleware que loga todas as requisicoes
    automaticamente.

    Adiciona:
    - trace_id: ID unico para rastrear a requisicao
    - latency_ms: Tempo de resposta em milissegundos
    - Headers de resposta com trace_id e tempo
    """

    async def dispatch(self, request: Request, call_next):
        # Gera trace_id unico para rastreamento
        # Permite correlacionar logs da mesma requisicao
        trace_id = str(uuid.uuid4())[:8]
        request.state.trace_id = trace_id
        # Captura tempo inicial
        start_time = time.perf_counter()
        # Processa a requisicao
        response = await call_next(request)

        # Calcula latencia
        latency_ms = (time.perf_counter() - start_time) *
1000

        # Log estruturado da requisicao
        logger.info(
            "request_completed",
            extra={
                "trace_id": trace_id,
                "method": request.method,
                "path": request.url.path,
```

```

        "status_code": response.status_code,
        "latency_ms": round(latency_ms, 2),
        "client_ip": request.client.host if
request.client else None,
    }
)
# Adiciona headers de rastreamento na resposta
# Uteis para debug do cliente
response.headers["X-Trace-ID"] = trace_id
response.headers["X-Response-Time-Ms"] =
str(round(latency_ms, 2))
return response

```

Código-fonte 2 – Middleware que automaticamente loga todas as requisições

Fonte: Elaborado pelo autor (2026)

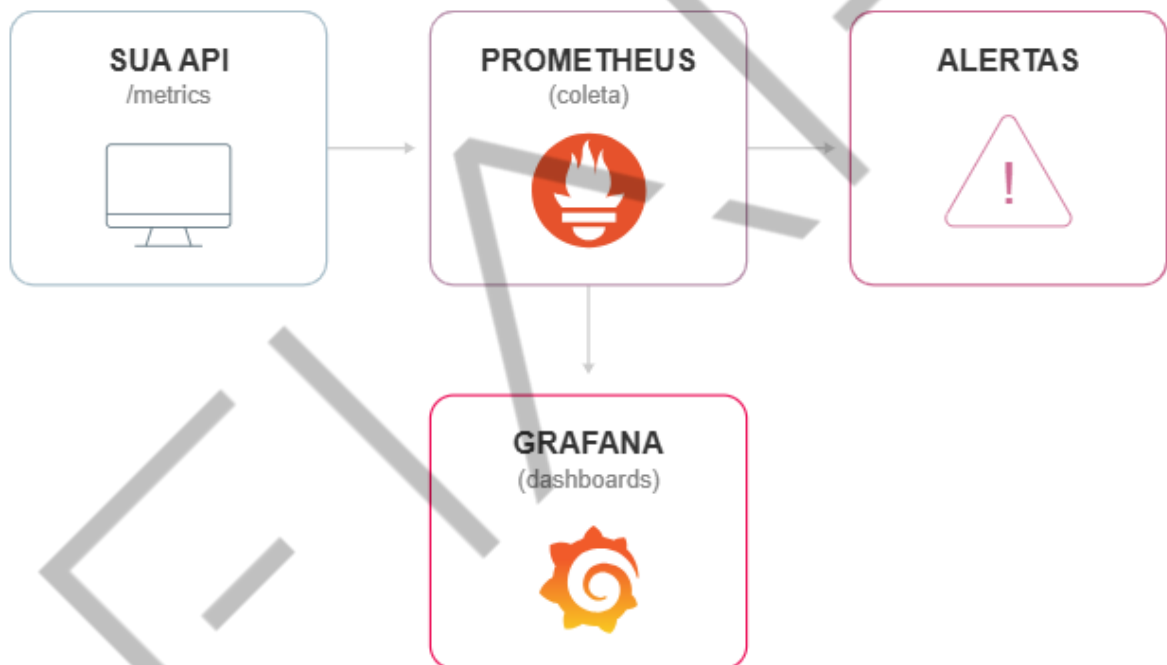


Figura 4 - Implementando Métricas com Prometheus

Fonte: Google Imagens (2026)

Vamos instalar as bibliotecas e criar o módulo de métricas customizadas: pip install prometheus-client prometheus-fastapi-instrumentator

```

"""
Metricas Customizadas com Prometheus
Define contadores, histogramas e gauges para monitoramento
"""
from prometheus_client import Counter, Histogram, Gauge

```

```
#
=====
# CONTADORES (Counter)
# Somam eventos que so aumentam (nunca diminuem)
#
=====
# Total de predicoes, com labels para classe e usuario
PREDICTIONS_TOTAL = Counter(
    'iris_predictions_total',
    'Total de predicoes realizadas',
    ['classe', 'user'] # Labels para filtrar no Prometheus
)
# Tentativas de login
LOGIN_ATTEMPTS = Counter(
    'login_attempts_total',
    'Total de tentativas de login',
    ['status'] # success ou failed
)
# Erros por tipo
ERRORS_TOTAL = Counter(
    'api_errors_total',
    'Total de erros',
    ['endpoint', 'error_type'])
#
=====
# HISTOGRAMAS (Histogram)
# Distribuicao de valores (ex.: latencia)
# Permite calcular percentis (p50, p95, p99)
#
=====
# Latencia das predicoes
PREDICTION_LATENCY = Histogram(
    'iris_prediction_latency_seconds',
    'Latencia das predicoes em segundos',
    buckets=[0.01, 0.025, 0.05, 0.1, 0.25, 0.5, 1.0, 2.5]
)
# Latencia geral das requisicoes
REQUEST_LATENCY = Histogram(
    'http_request_latency_seconds',
    'Latencia das requisicoes HTTP',
    ['method', 'endpoint'],
    buckets=[0.01, 0.05, 0.1, 0.25, 0.5, 1.0, 2.5, 5.0]
)
=====
# GAUGES (Gauge)
# Valores que podem subir e descer (estado atual)
```

```
#
=====
# Modelo carregado? (1 = sim, 0 = nao)
MODEL_LOADED = Gauge(
    'model_loaded',
    'Indica se o modelo esta carregado (1) ou nao (0)'
)
# Confianca media das ultimas predicoes
AVG_CONFIDENCE = Gauge(
    'prediction_avg_confidence',
    'Confianca media das ultimas predicoes'
)

Código-fonte 3 – Módulo de métricas customizadas
Fonte: Elaborado pelo autor (2026)
```

Vamos criar o arquivo de configuração prometheus.yml:

```
aula_07 > prometheus > ! prometheus.yml
1  # Configuracao do Prometheus
2  # Define de onde coletar metricas (scrape)
3
4  global:
5      scrape_interval: 15s      # Coleta metricas a cada 15 segundos
6      evaluation_interval: 15s  # Avalia regras de alerta a cada 15 segundos
7
8  # Regras de alerta (arquivo separado)
9  rule_files:
10     - "alerts.yml"
11
12  # Alvos de scraping
13  scrape_configs:
14      # Prometheus monitora a si mesmo
15      - job_name: 'prometheus'
16        static_configs:
17          - targets: ['localhost:9090']
18
19      # Nossa API Iris
20      - job_name: 'api-iris'
21        static_configs:
22          - targets: ['api:8000'] # Nome do servico no docker-compose
23        metrics_path: '/metrics' # Endpoint que expoe metricas
24
```

Figura 5 - Arquivo de configuração prometheus.yml
Fonte: Elaborado pelo autor (2026)

Alertas

Monitorar métricas e logs é ótimo, mas a verdadeira tranquilidade operacional vem de configurar alertas proativos. Alertas são notificações automáticas (e-mail, mensagem no Slack, SMS ou integração com pager) disparadas quando alguma condição anômala ocorre no sistema. A ideia é que a equipe seja avisada imediatamente ao surgirem sinais de problema, em vez de descobrir horas depois por reclamação de usuários.

Deve-se evitar alertas demais (para não causar “alert fatigue”), mas sim focar em condições críticas que exigem intervenção. Exemplos de alertas básicos em APIs de ML:

- **Erro 5xx Elevado:** por exemplo, configure um alerta para ser disparado se a taxa de erros do tipo 5xx (erros de servidor) ultrapassar 5% de todas as requisições em um período de 5 minutos. Essa regra ajuda a identificar falhas graves. Usando ferramentas como Prometheus/Alertmanager, você pode usar uma fórmula (como `rate(http_requests_total{status=~"5.."}[5m]) > 0.05`) para monitorar isso e disparar um alerta se a condição persistir por alguns minutos. Houve um caso real onde um alerta de 5% de erros em 2 minutos avisou a equipe antes dos clientes, permitindo uma correção rápida.
- **Latência Alta e Constante:** um alerta deve ser acionado se o tempo de resposta p95 (o tempo que 95% das requisições levam) permanecer acima de um limite (ex.: mais de 1 segundo) por um período de tempo definido (N minutos). Isso indica uma possível degradação do desempenho ou problemas de timeout com serviços externos.
- **Queda de throughput ou disponibilidade:** se o número de requisições cair abruptamente a zero em horário de pico, ou se o serviço não responder (downtime), deve-se alertar – pode indicar uma interrupção completa.
- **Falha em componentes auxiliares:** por exemplo, falta de espaço em disco, fila de processamento estagnada ou falha na coleta de métricas (se Prometheus não conseguir scrapear etc.). Em sistemas de ML, poderíamos ter um alerta se o serviço de inferência não conseguir carregar o modelo (indicando modelo corrompido ou path errado).

```
aula_07 > prometheus > ! alerts.yml
1  # Regras de Alerta do Prometheus
2
3  groups:
4    - name: api-iris-alerts
5      rules:
6        # Alerta: API fora do ar
7        - alert: APIDown
8          expr: up{job="api-iris"} == 0
9          for: 1m
10         labels:
11           severity: critical
12         annotations:
13           summary: "API Iris esta fora do ar"
14           description: "A API nao responde ha mais de 1 minuto"
15
16        # Alerta: Taxa de erros alta
17        - alert: HighErrorRate
18          expr: |
19            sum(rate(http_requests_total{status=~"5.."}[5m]))
20            / sum(rate(http_requests_total[5m])) > 0.05
21          for: 2m
22          labels:
23            severity: high
24          annotations:
25            summary: "Taxa de erros acima de 5%"
26            description: "{{ $value | humanizePercentage }}" das requisicoes estao falhando"
27
28        # Alerta: Latencia alta
29        - alert: HighLatency
30          expr: |
31            histogram_quantile(0.95,
32              rate(http_request_latency_seconds_bucket[5m])
33            ) > 1
34          for: 5m
35          labels:
36            severity: medium
37          annotations:
38            summary: "Latencia p95 acima de 1 segundo"
39            description: "Latencia atual: {{ $value | humanizeDuration }}"
40
41        # Alerta: Modelo nao carregado
42        - alert: ModelNotLoaded
43          expr: model_loaded == 0
44          for: 1m
45          labels:
46            severity: high
47          annotations:
48            summary: "Modelo de ML nao esta carregado"
49            description: "A API esta funcionando mas sem o modelo de predicao"
50
```

Figura 6 - Regras de alerta (Prometheus)

Fonte: Elaborado pelo autor (2026)

Docker Compose: Orquestração de Containers para Monitoramento de APIs

Enquanto o Docker permite a containerização individual da sua API, a monitorização eficaz exige a execução coordenada de múltiplos serviços, como:

- **API:** sua aplicação FastAPI.
- **Prometheus:** coleta as métricas de desempenho.
- **Grafana:** visualiza as métricas em dashboards interativos.

Executar esses serviços separadamente seria ineficiente.

O Docker Compose simplifica essa orquestração. É uma ferramenta que utiliza um único arquivo YAML (docker-compose.yml) para definir e gerenciar o ciclo de vida (subir, parar etc.) de todos os containers necessários. Em vez de múltiplos comandos, você precisa apenas de um comando simples para iniciar todo o ambiente.

SERVIÇO	ORIGEM	FUNÇÃO
API	build: . (Seu Dockerfile)	Executa sua aplicação FastAPI.
Prometheus	image: prom/prometheus (Docker Hub)	Coleta métricas do endpoint /metrics.
Grafana	image: grafana/grafana (Docker Hub)	Oferece dashboards visuais para as métricas.

Tabela 2 - Estrutura do docker-compose.yml (Visão Geral dos Serviços):
Fonte: Elaborado pelo autor (2026)

Nota Importante: Prometheus e Grafana são imagens pré-construídas disponíveis no Docker Hub, não fazendo parte do código da sua aplicação. O Docker Compose apenas gerencia como esses serviços se conectam entre si e com sua API.

COMANDO	FUNÇÃO
<code>docker-compose up -d</code>	Inicia todos os containers em segundo plano (modo detached).
<code>docker-compose ps</code>	Exibe o status atual de todos os containers definidos.
<code>docker-compose logs api</code>	Exibe os logs gerados especificamente pelo container da API.
<code>docker-compose down</code>	Para e remove todos os containers e redes associadas.

Tabela 3 - Comandos Essenciais do Docker Compose:
Fonte: Elaborado pelo autor (2026)

Demonstração Rápida (Monitoramento Completo)

Com a API devidamente instrumentada (via `metrics.py`), a execução do Docker Compose levanta toda a infraestrutura de monitoramento.

Após a execução, os seguintes serviços estarão disponíveis:

SERVIÇO	URL	DESCRIÇÃO
API (Swagger)	http://localhost:8000/docs	Documentação interativa da API.
API (Métricas)	http://localhost:8000/metrics	Endpoint das métricas no formato Prometheus.
Prometheus	http://localhost:9090	Interface para consultas e alertas de métricas.
Grafana	http://localhost:3000	Interface de Dashboards (Login Padrão: admin/admin).

Tabela 4 - Serviços disponíveis
Fonte: Elaborado pelo autor (2026)

O arquivo final docker-compose.yml ficará assim:

```
#
=====
# Docker Compose - Stack de Monitoramento
# Roda: API + Prometheus + Grafana
# Uso: docker-compose up -d
#
=====
services:
  # API FastAPI com metricas
  api:
    build: .
    container_name: aula_07_api
    ports:
      - "8000:8000"
```

```
environment:
  - ENVIRONMENT=production
  - JWT_SECRET=sua-chave-secreta-aqui
  - TOKEN_EXPIRE_MINUTES=60
healthcheck:
  test: ["CMD", "curl", "-f",
"http://localhost:8000/health"]
  interval: 10s
  timeout: 5s
  retries: 3
  start_period: 10s
networks:
  - monitoring

# Prometheus - Coleta metricas
prometheus:
  image: prom/prometheus:v2.47.0
  container_name: aula_07_prometheus
  ports:
    - "9090:9090"
  volumes:
    -
./prometheus/prometheus.yml:/etc/prometheus/prometheus.yml
    -
./prometheus/alerts.yml:/etc/prometheus/alerts.yml
  - prometheus_data:/prometheus
  command:
    - '--config.file=/etc/prometheus/prometheus.yml'
    - '--storage.tsdb.path=/prometheus'
  networks:
    - monitoring

# Grafana - Visualiza metricas
grafana:
  image: grafana/grafana:10.2.0
  container_name: aula_07_grafana
  ports:
    - "3000:3000"
  environment:
    - GF_SECURITY_ADMIN_PASSWORD=admin
    - GF_SECURITY_ADMIN_USER=admin
  volumes:
    - grafana_data:/var/lib/grafana
  networks:
    - monitoring
  depends_on:
    - prometheus

# Volumes persistentes
volumes:
  prometheus_data:
```

```
grafana_data:

# Rede para comunicacao entre containers
networks:
  monitoring:
    driver: bridge
```

Código-fonte 4 – Arquivo final docker-compose.yml
Fonte: Elaborado pelo autor (2026)

Por fim, vamos atualizar o nosso arquivo main.py com a inclusão dos logs estruturados, métricas via prometheus e middleware de logging.

```
main.py :
"""
API Iris com JWT - Versao com Monitoramento Completo
Inclui: Logs estruturados, Metricas Prometheus, Middleware
de logging
"""
import os
import pickle
import time
from pathlib import Path

import numpy as np
from fastapi import Depends, FastAPI, HTTPException,
Request
from pydantic import BaseModel, Field

# Prometheus
from prometheus_fastapi_instrumentator import import
Instrumentator

# Modulos locais
from app.auth import (
    create_token,
    get_current_user,
    authenticate_user,
    TOKEN_EXPIRE_MINUTES,
)
from app.logging_config import setup_logging
from app.middleware import LoggingMiddleware
from app.metrics import (
    PREDICTIONS_TOTAL,
    LOGIN_ATTEMPTS,
    PREDICTION_LATENCY,
    MODEL_LOADED,
)
```

```
#
=====
=====
# CONFIGURACOES
#
=====
=====
ENVIRONMENT = os.getenv("ENVIRONMENT", "development")
API_VERSION = "2.1.0" # Versao atualizada com monitoramento

#
=====
=====
# LOGGING ESTRUTURADO
#
=====
=====
logger = setup_logging(os.getenv("LOG_LEVEL", "INFO"))

#
=====
=====
# SCHEMAS
#
=====
=====
class LoginRequest(BaseModel):
    username: str
    password: str

class TokenResponse(BaseModel):
    access_token: str
    token_type: str = "bearer"
    expires_in: int

class IrisRequest(BaseModel):
    sepal_length: float = Field(..., ge=0, le=10,
description="Comprimento da sepala (cm)")
    sepal_width: float = Field(..., ge=0, le=10,
description="Largura da sepala (cm)")
    petal_length: float = Field(..., ge=0, le=10,
description="Comprimento da petala (cm)")
    petal_width: float = Field(..., ge=0, le=10,
description="Largura da petala (cm)")
```

```
class IrisResponse(BaseModel):
    sucesso: bool
    classe: str
    probabilidades: dict
    usuario: str

#
=====
# CARREGAMENTO DO MODELO
#
=====
=====
MODEL_PATHS = [
    Path("app/models/modelo_iris.pkl"),
    Path("/app/app/models/modelo_iris.pkl"),
    Path("modelo_iris.pkl"),
]

modelo = None
classes = None

for model_path in MODEL_PATHS:
    if model_path.exists():
        with open(model_path, 'rb') as f:
            modelo = pickle.load(f)
            classes_path = model_path.parent /
"classes_iris.pkl"
            if classes_path.exists():
                with open(classes_path, 'rb') as f:
                    classes = pickle.load(f)
            logger.info("model_loaded", extra={"path":
str(model_path)})
            break

MODELO_OK = modelo is not None and classes is not None

# Atualiza metrica do Prometheus
MODEL_LOADED.set(1 if MODELO_OK else 0)

if not MODELO_OK:
    logger.warning("model_not_found",
extra={"searched_paths": [str(p) for p in MODEL_PATHS]})

#
=====
# APLICACAO FASTAPI
```

```
#
=====
app = FastAPI(
    title="API Iris com Monitoramento",
    description="API de classificacao com logs estruturados
e metricas Prometheus.",
    version=API_VERSION,
    docs_url="/docs",
    redoc_url="/redoc",

    redoc_js_url="https://cdn.jsdelivr.net/npm/redoc@2/bundles/red
oc.standalone.js",
)

# Adiciona Middleware de Logging (intercepta todas as
requisicoes)
app.add_middleware(LoggingMiddleware)

# Instrumentacao Prometheus (adiciona metricas automaticas
+ endpoint /metrics)
Instrumentator().instrument(app).expose(app,
endpoint="/metrics")

#
=====
# ENDPOINTS
#
=====
@app.get("/")
def home():
    """Informacoes da API."""
    return {
        "api": "Iris Classifier",
        "versao": API_VERSION,
        "ambiente": ENVIRONMENT,
        "modelo_carregado": MODELO_OK,
        "docs": "/docs",
        "metrics": "/metrics",
        "health": "/health",
    }

@app.get("/health")
def health():
    """Health check para monitoramento."""
    return {
        "status": "healthy" if MODELO_OK else "degraded",
```

```
        "modelo": MODELO_OK,
        "ambiente": ENVIRONMENT,
        "version": API_VERSION,
    }

@app.post("/login", response_model=TokenResponse)
def login(credentials: LoginRequest, request: Request):
    """Faz login e retorna token JWT."""
    user = authenticate_user(credentials.username,
credentials.password)
    trace_id = getattr(request.state, 'trace_id', 'N/A')

    if not user:
        # Metrica: login falhou
        LOGIN_ATTEMPTS.labels(status="failed").inc()

        # Log estruturado
        logger.warning("login_failed", extra={
            "username": credentials.username,
            "trace_id": trace_id,
        })
        raise HTTPException(status_code=401,
detail="Usuario ou senha incorretos")

    # Metrica: login sucesso
    LOGIN_ATTEMPTS.labels(status="success").inc()

    token = create_token(user["username"], user["role"])

    # Log estruturado
    logger.info("login_success", extra={
        "username": user["username"],
        "role": user["role"],
        "trace_id": trace_id,
    })

    return TokenResponse(access_token=token,
expires_in=TOKEN_EXPIRE_MINUTES * 60)

@app.get("/me")
def get_me(current_user: dict = Depends(get_current_user)):
    """Retorna informacoes do usuario logado."""
    return current_user

@app.post("/predict", response_model=IrisResponse)
def predict(payload: IrisRequest, request: Request,
current_user: dict = Depends(get_current_user)):
    """Faz predicao (requer autenticacao)."""
```

```
if not MODELO_OK:
    raise HTTPException(status_code=503,
detail="Modelo nao disponivel")

trace_id = getattr(request.state, 'trace_id', 'N/A')

# Mede latencia da predicao
start = time.perf_counter()

features = np.array([[
    payload.sepal_length, payload.sepal_width,
    payload.petal_length, payload.petal_width
]])

pred_idx = modelo.predict(features)[0]
probs = modelo.predict_proba(features)[0]
classe = classes[pred_idx]
confidence = float(max(probs))

latency = time.perf_counter() - start

# Metricas Prometheus
PREDICTIONS_TOTAL.labels(classe=classe,
user=current_user["username"]).inc()
PREDICTION_LATENCY.observe(latency)

# Log estruturado
logger.info("prediction_completed", extra={
    "trace_id": trace_id,
    "user": current_user["username"],
    "classe": classe,
    "confidence": round(confidence, 4),
    "latency_ms": round(latency * 1000, 2),
    "features": {
        "sepal_length": payload.sepal_length,
        "sepal_width": payload.sepal_width,
        "petal_length": payload.petal_length,
        "petal_width": payload.petal_width,
    }
})

return IrisResponse(
    sucesso=True,
    classe=classe,
    probabilidades={classes[i]: round(float(p), 4) for
i, p in enumerate(probs)},
    usuario=current_user["username"]
)
```

Código-fonte 5 – Arquivo main.py com logs estruturados, métricas via prometheus e middleware de logging

Fonte: Elaborado pelo autor (2026)

Vamos buildar nosso docker-compose e testar a API!

```
PS C:\B3-API> cd ./aula_07
PS C:\B3-API\aula_07> docker-compose up -d
[+] Running 4/4
✓ Network aula_07_monitoring Created 0.1s
✓ Container aula_07_api Started 1.1s
✓ Container aula_07_prometheus Started 1.1s
✓ Container aula_07_grafana Started 1.2s
PS C:\B3-API\aula_07> docker-compose ps
NAME                IMAGE                COMMAND                  SERVICE    CREATED         STATUS                                     PORTS
aula_07_api          aula_07-api          "uvicorn app.main:ap..." api        9 seconds ago   Up 8 seconds (health: starting)         0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp
aula_07_grafana      grafana/grafana:10.2.0 "/run.sh"              grafana    9 seconds ago   Up 8 seconds                           0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp
aula_07_prometheus   prom/prometheus:v2.47.0 "/bin/prometheus --c..." prometheus  9 seconds ago   Up 8 seconds                           0.0.0.0:9090->9090/tcp, [::]:9090->9090/tcp
```

Figura 7 – Build do docker-compose e teste da API
Fonte: Elaborado pelo autor (2026)

No swagger agora conseguimos ver nosso endpoint /metrics que serve o Prometheus:

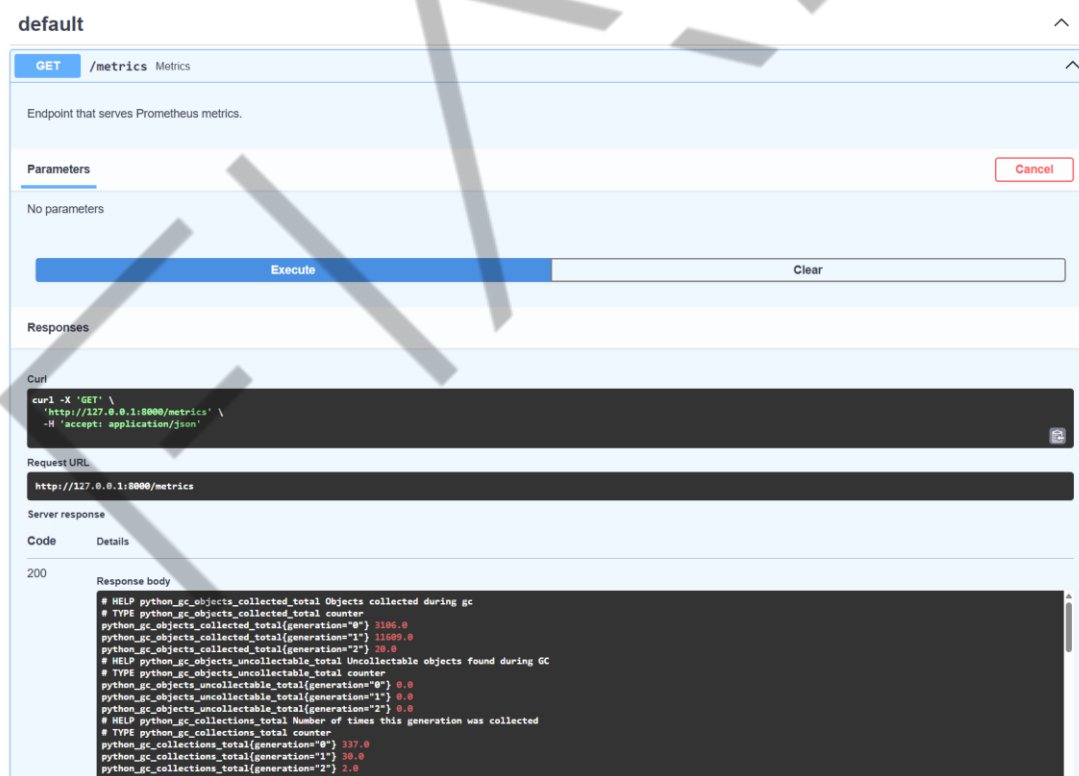


Figura 8 - endpoint /metrics
Fonte: Elaborado pelo autor (2026)

Podemos acessar o Prometheus via <http://localhost:9090>

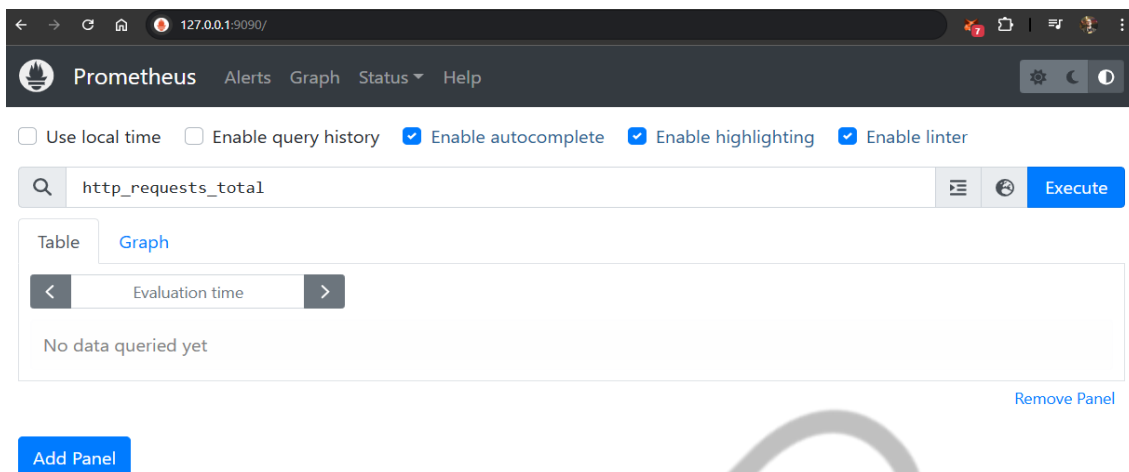


Figura 9 - Prometheus via <http://localhost:9090>
Fonte: Elaborado pelo autor (2026)

Aqui já podemos visualizar diversas métricas e criar nossos painéis, inclusive criar alguns gráficos básicos para visualização rápida. Mas, para a produção real, o ideal é utilizarmos o Grafana onde podemos compartilhar dashboards com o time inteiro, com diversas fontes de dados e múltiplas APIs no mesmo dashboard.

Vamos acessar o Grafana via: <http://localhost:3000> e fazer o login com admin/admin para testes.

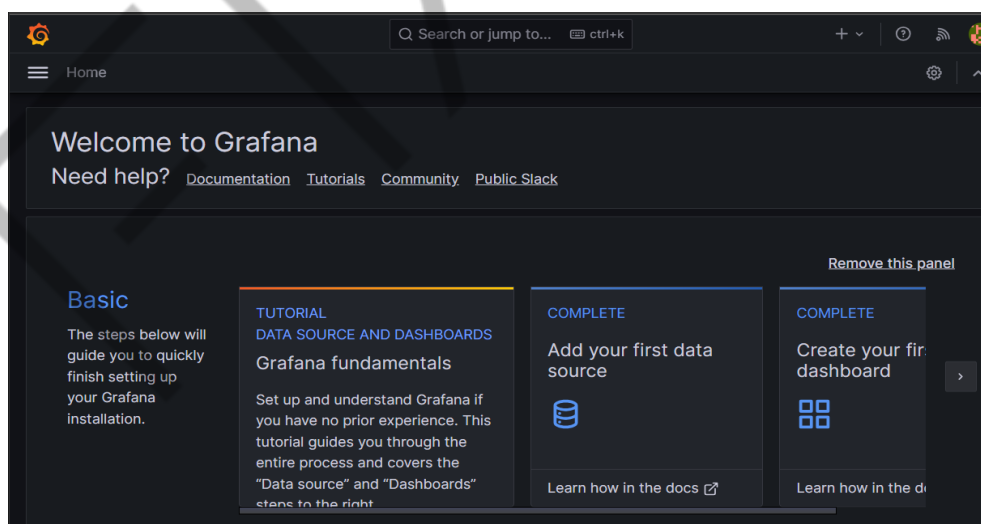


Figura 9 - Grafana via: <http://localhost:3000>
Fonte: Elaborado pelo autor (2026)

Na parte de conexões, vamos adicionar uma nova Data Source do Prometheus:

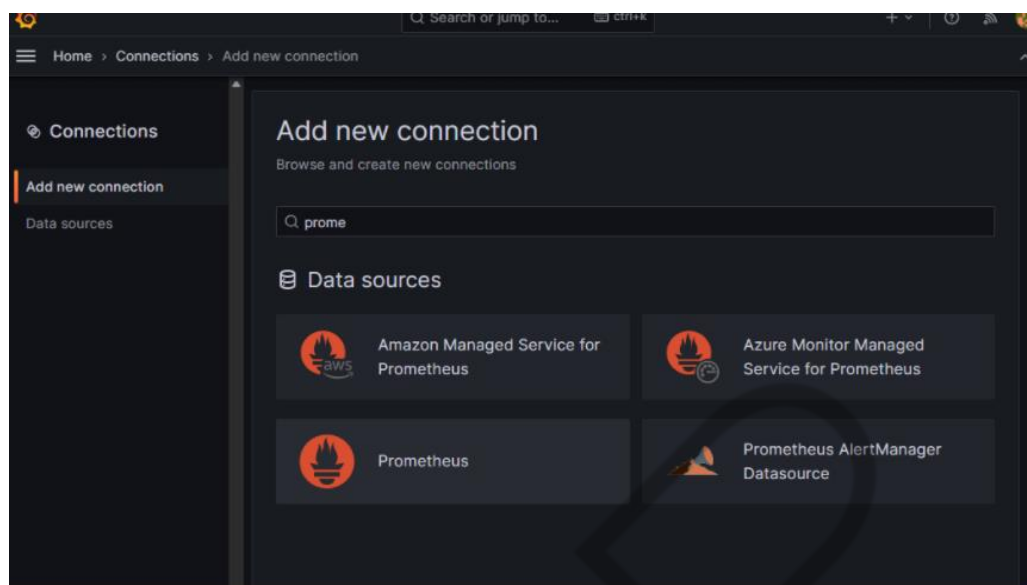


Figura 10 - Nova Data Source do Prometheus
Fonte: Elaborado pelo autor (2026)

Preenchemos o campo Connection com o URL do Prometheus:

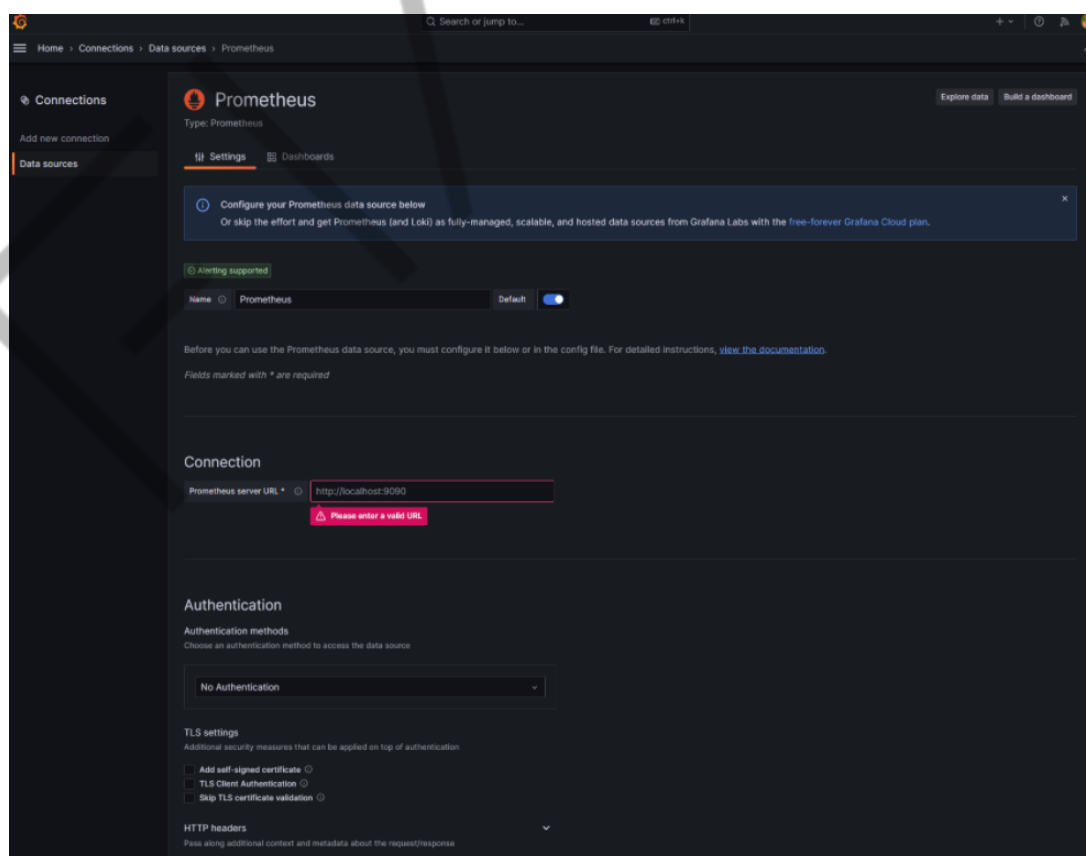


Figura 11 - Campo Connection com o URL do Prometheus
Fonte: Elaborado pelo autor (2026)

E, por fim, podemos criar um novo dashboard para monitoramento de métricas, como o número total de requisições!

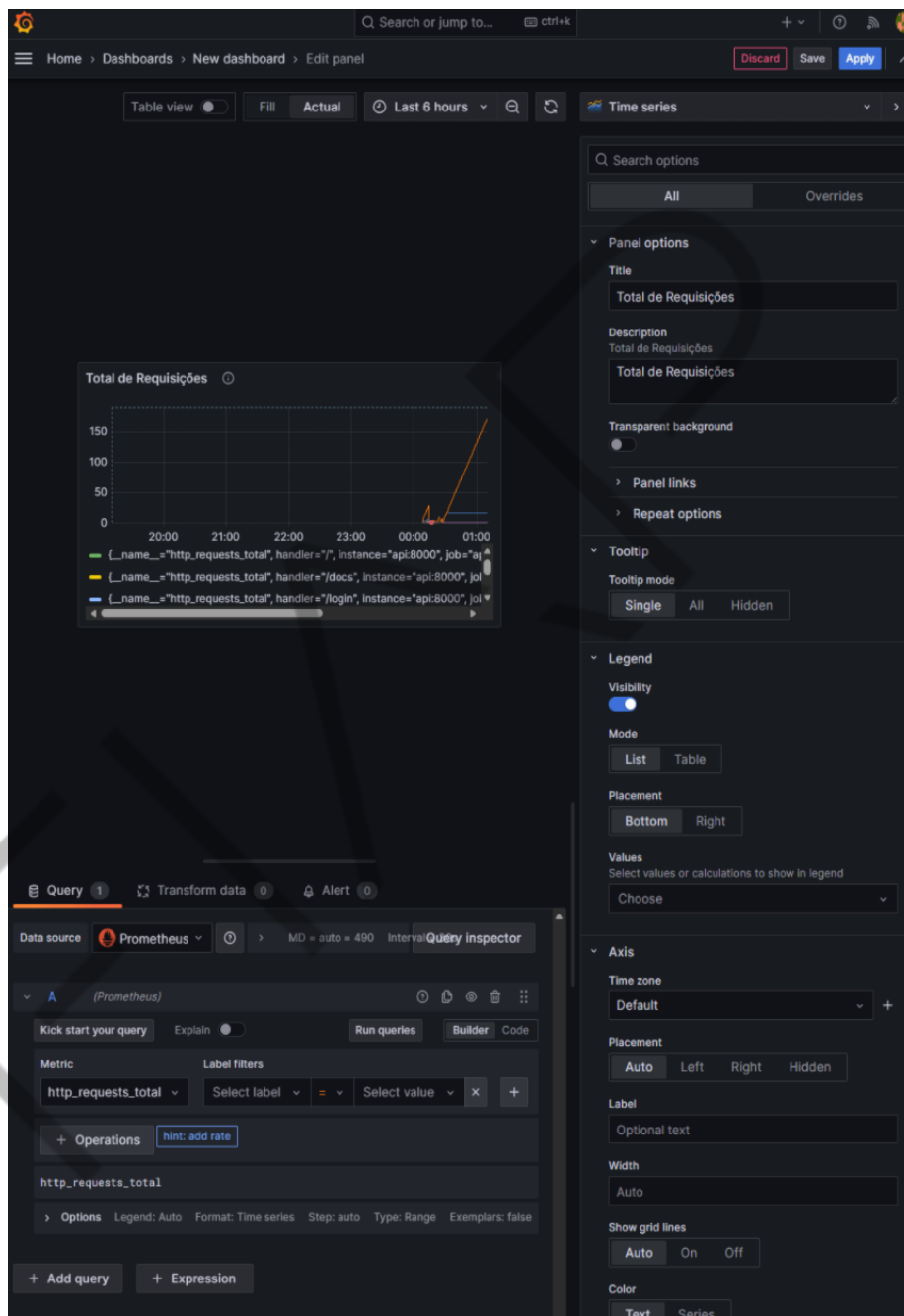


Figura 12 - Número total de requisições
Fonte: Elaborado pelo autor (2026)

Agora temos um sistema de monitoramento de métricas inicial em nossa API!

Versionamento e Atualização de Modelos de Machine Learning em APIs

A natureza dos modelos de ML exige manutenção e atualização contínuas, pois dados, comportamento do usuário e algoritmos evoluem. O desafio é realizar essas atualizações em uma API de inferência sem interromper o serviço.

A necessidade de atualização é geralmente sinalizada pela degradação do desempenho preditivo (drift), indicando que o modelo está cometendo mais erros (identificados por feedback ou análises offline). Mudanças no contexto de negócios ou na distribuição dos dados de entrada também sugerem a necessidade de retreino, mesmo sem métricas de acurácia explícitas. A prática de MLOps recomenda reavaliações periódicas (ex.: retreino mensal ou trimestral) e atualizações para corrigir bugs (ex.: bias indesejado) ou incorporar melhorias algorítmicas.

A substituição direta de um modelo em produção é arriscada. Recomenda-se o uso de estratégias de deployment que minimizem o risco de interrupção do serviço:

- **Versionamento de Endpoints:**
 - Consiste em manter o modelo antigo em um endpoint versionado (ex.: `/api/v1/predict`) e disponibilizar a nova versão em um novo endpoint (ex.: `/api/v2/predict`).
 - Ambas as versões coexistem por um período. Os clientes migram gradualmente ou testam a nova versão, apontando para a nova URL.
 - Essa abordagem garante que os consumidores da API só usem o modelo novo quando estiverem prontos, além de facilitar o rollback imediato para a versão anterior em caso de problemas.
- **Blue-Green Deployment:**
 - Mantém-se dois ambientes idênticos: o Blue (versão atual em produção, ex.: **v1**) e o Green (nova versão preparada, ex.: **v2**).
 - O tráfego é atendido pelo ambiente Blue. A versão Green é testada em paralelo, sem tráfego público.
 - Após a validação, o tráfego é instantaneamente redirecionado para o Green (que se torna "prod") através da alteração do balanceador de carga ou DNS.

- O ambiente Blue permanece inativo, servindo como uma opção de fallback imediato. Oferece implantações sem downtime e com rollback instantâneo.
- **Canary Release:**
 - É uma liberação mais gradual. Apenas uma pequena parcela do tráfego (ex.: 5% — o "canário") é direcionada para a nova versão, enquanto a maioria (ex.: 95%) permanece na versão estável.
 - A nova versão é monitorada de perto. Se as métricas forem positivas, a porcentagem de tráfego é aumentada gradativamente (ex.: 20%, 50%, até 100%).
 - Em caso de problemas (aumento de erros, piora de latência), o tráfego é abortado e totalmente redirecionado para a versão estável anterior.
 - Essa estratégia minimiza o impacto de bugs, afetando apenas uma pequena fração dos usuários inicialmente. Permite essencialmente um Teste A/B em produção para comparar os resultados do modelo novo (ex.: aumento de conversão).

Importância do Monitoramento e Versionamento Interno

Em todas as estratégias de deployment, o monitoramento rigoroso é crucial. Durante a implantação de um modelo novo, deve-se acompanhar atentamente métricas de erro, latência e a distribuição dos resultados do modelo, pois variações podem impactar sistemas a jusante (downstream).

Além do versionamento da API, é fundamental o controle de versão dos artefatos do modelo (ex.: "modelo fraude v1.3"), utilizando versionamento semântico. Metadados da versão devem ser armazenados junto com as previsões (ex.: registrar qual versão do modelo gerou cada resposta), garantindo rastreabilidade e facilitando auditorias e o rastreamento de respostas afetadas por bugs descobertos posteriormente.

Checklist de Manutenção Contínua de APIs em Produção

A manutenção de uma API em produção exige disciplina e proatividade para garantir sua confiabilidade, performance e segurança.

I. Monitoramento e Resposta a Incidentes

- **Monitoramento Ativo Diário:** inspecionar regularmente dashboards de métricas e logs. O foco é identificar anomalias e assegurar que alertas críticos (ex.: erros 5xx, downtime) não estejam ativos. Analisar amostras de logs de erro para detectar padrões emergentes
- **Planos de Contingência e Testes:** manter e testar procedimentos de failover e rollback para garantir a rápida recuperação. Simular cenários de falha (ex.: indisponibilidade de nó) para confirmar o correto redirecionamento pelo balanceador de carga. Testar a recuperação de backups de modelos antigos sob pressão para validar a funcionalidade de rollback de modelo.
- **Reação a Alertas:** confirmar que os alertas estão configurados corretamente e que a equipe consegue reagir dentro dos tempos de resposta esperados (SLOs).

II. Performance e Estabilidade

- **Testes de Carga Regulares:** agendar testes de desempenho (ex.: mensalmente) simulando o volume de requisições de produção. O objetivo é identificar regressões de performance ou vazamentos de memória causados por sucessivos deploys e garantir que a API continue a atender os SLOs definidos.
- **Backups e Arquivamento:** confirmar a execução correta de backups automatizados. Isso inclui o armazenamento de modelos versionados (com testes de restore validados) e o arquivamento de logs conforme a política. É mandatório possuir, no mínimo, um backup off-site (externo) para recuperação em caso de desastre.

III. Segurança e Conformidade

- **Revisões de Segurança e Pentests:** realizar periodicamente pentests (testes de intrusão) e análises de vulnerabilidade. Focar em autenticação, autorização e proteção contra input malicioso (aderência ao OWASP Top 10 para APIs).
- **Fortalecimento das Configurações:** revisar periodicamente configurações de segurança (certificados, tokens de acesso). Implementar recomendações de hardening (ex.: endpoints somente HTTPS, rotação de tokens).
- **Prevenção de Vazamento de Dados:** assegurar que os endpoints de inferência e os logs não estejam expondo dados sensíveis nas respostas ou nos registros.

IV. Atualização e Documentação

- **Atualizações e Patches:** manter a base de código, as dependências e as bibliotecas utilizadas (ex.: FastAPI, frameworks de ML) atualizadas. Aplicar patches de segurança imediatamente.
- **Documentação e Conhecimento Compartilhado:** atualizar a documentação sempre que houver mudanças significativas (novos endpoints, parâmetros ou esquemas de autenticação).
- **Lições Aprendidas e Treinamento:** compartilhar as lições de incidentes (post-mortems) e revisar os procedimentos on-call. Treinar novos membros da equipe na operação da API e no uso das ferramentas de monitoramento.

Com todas ferramentas que aprendemos e esse checklist em mente, podemos agora versionar a nossa API para uma solução completa!

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, exploramos como monitorar e manter APIs de inferência de ML em produção de forma profissional. Usando logs estruturados e métricas, obtemos observabilidade total do sistema, desde rastrear uma requisição específica nos logs até visualizar gráficos.

Adotamos ferramentas como Prometheus e Grafana para automatizar a coleta e visualização desses dados, estabelecendo dashboards e alertas. Discutimos também estratégias de deploy contínuo seguro com versionamento para atualizar modelos de ML minimizando riscos e interrupções.

Ao consolidar essas práticas, equipes de engenheiros(as) de Machine Learning podem operar serviços de IA em produção com a confiança de que problemas serão rapidamente detectados e resolvidos.

REFERÊNCIAS

GÉRÓN, A. **Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow**. Beijing: O'Reilly Media, 2023.

GIFT, N.; DEZA, A. **Practical MLOps**: Operationalizing Machine Learning Models. Sebastopol: O'Reilly Media, 2021.

LUBANOVIC, B. **FastAPI**: Modern Python Web Development. Sebastopol: O'Reilly Media, 2023.

NEWMAN, S. **Building Microservices**: Designing Fine-Grained Systems. Sebastopol: O'Reilly Media, 2021.

PALAVRAS-CHAVE

JWT. Token. API. WEB HTTP. REST. API RESTful. GraphQL. gRPC. JSON. MLOps.

EMSE

EMSE